

FULL-STACK ENTERPRISE SOFTWARE PROJECT

MCA WALLAH

Personal Portfolio & Dynamic CMS System

A high-performance, real-time developer showcase platform and content management architecture engineered with React 19, Vite 8, Node.js, Express, and MongoDB.

PROJECT TYPE

Production Web Application & CMS

DOCUMENTATION DATE

August 17, 2026

CORE FRONTEND STACK

React 19.2.4 • Vite 8 • CSS Tokens

VERSION

v1.0.0 (Production Release)

PRIMARY AUTHOR / DEVELOPER

Mahadeb Maity

CORE BACKEND STACK

Node.js 18+ • Express 4.21.2 • MongoDB 8

02 Project Overview

MCA WALLAH Portfolio & Dynamic CMS is an advanced, production-grade developer portfolio website tightly integrated with a secure, full-scale administrative Content Management System (CMS). Unlike conventional static portfolio sites that require developers to manually edit JSX/HTML code, push commits, and redeploy every time a project, skill, resume, or contact link changes, this system decouples content and presentation logic through an intuitive visual administration studio.

The platform is designed to serve two primary audiences: **(1) Public Visitors & Recruiters**, who experience a fast, responsive, glassmorphic portfolio equipped with real-time interactive themes, multi-resume downloads, contact dispatching, and an embedded arcade lounge; and **(2) The Portfolio Owner / Admin**, who maintains complete programmatic control over every single visual component, section order, layout variant, database asset, and live telemetry log without writing a single line of backend code.

03 Problem Statement

In modern software engineering recruitment and freelance markets, static single-page portfolios suffer from major technical and operational bottlenecks:

- **High Maintenance Friction:** Updating resume versions, project descriptions, live deployment links, or skill proficiencies requires source code modification, recompilation, and production deployment cycles.
- **Single-Resume Rigidity:** Standard portfolios provide only a single generic resume link. Recruiters targeting specialized roles (e.g., Full Stack Engineer vs. UI/UX Designer vs. Python Specialist) require role-tailored resumes.
- **Zero Interaction Observability:** Developers typically have no insight into who downloaded their resume, which projects generated clicks, or how visitors engaged with the site beyond generic third-party analytics scripts.
- **Monolithic Aesthetics:** Traditional portfolios lock the user into a single layout and color palette, preventing personalization or dynamic theme adaptation.

The **MCA WALLAH system resolves these challenges** by implementing a dynamic, database-driven architecture featuring an administrative CRUD studio, multi-resume vault management, custom background styling engine, telemetry logging, and interactive visitor engagement modules.

04 Project Objectives

Primary Objectives

- Develop a fast, modern, mobile-responsive Single Page Application using React 19 and Vite 8.
- Construct a resilient RESTful backend using Node.js, Express, and MongoDB Mongoose.
- Build a secure, authenticated Admin CMS Studio with JWT-based authorization and BCrypt password encryption.
- Provide real-time content editing for all portfolio sections without code changes or restarts.
- Implement a Multi-Resume Vault allowing concurrent activation of role-specific resumes.

Secondary Objectives

- Build an interactive Developer Gaming Arcade (Snake, 2048, Typing Challenge, Tic-Tac-Toe) with MongoDB leaderboard persistence.
- Implement cross-tab instant state synchronization using the browser `BroadcastChannel` API.
- Deliver a Draggable Floating Background Widget supporting 8 backdrop presets and custom HSL color picking.
- Provide telemetry tracking, visitor analytics, and automated Nodemailer contact notifications.

- Deliver full database JSON backup, export, and restore capabilities.

05 Key Features Matrix

Feature Name	Description	User Role	Status
Dynamic Navigation Dock	4 selectable presentation layouts (Floating Dock, Cyber Capsule, Minimal Island, Full-Width Bar) with customizable brand logo, live pulse, and CTA button.	Visitor / Admin	IMPLEMENTED
Multi-Style Hero Studio	4 hero presets (Glassmorphism, Split Code IDE, Editorial Spotlight, Cyber Hologram) with dynamic typewriter roles, tech badges, and draggable backdrop widget.	Visitor / Admin	IMPLEMENTED
Multi-Resume Vault	Permanent document storage supporting multiple categorized resume uploads with custom display titles and multi-button download dropdowns.	Visitor / Admin	IMPLEMENTED
Categorized Projects Showcase	Filterable project cards with live demo triggers, GitHub repository links, star/fork metrics, and tag filtering.	Visitor / Admin	IMPLEMENTED
Interactive Gaming Arcade	4 built-in retro developer mini-games (Snake, 2048, Typing WPM Challenge, Minimax Tic-Tac-Toe) with dedicated standalone mode (<code>/arcade</code>) and global leaderboards.	Visitor / Player	IMPLEMENTED
Contact & Newsletter Dispatcher	Rate-limited contact messaging with automated Nodemailer email routing, IP logging, and newsletter subscription engine.	Visitor / Admin	IMPLEMENTED
Admin CMS Studio	Password-protected administrative dashboard with full CRUD capabilities over all 11 sections, message inbox, subscriber lists, and user role management.	Admin	IMPLEMENTED
Telemetry & Activity Logging	Audit trail recording user signups, logins, resume downloads, game plays, button clicks, and admin mutations.	Admin	IMPLEMENTED
JSON Database Backup & Restore	Complete JSON snapshot export and one-click data restore mechanism for instant disaster recovery.	Admin	IMPLEMENTED

06 User Roles and Permissions

User Role	Authentication Method	Accessible Modules	System Restrictions
Public Visitor / Recruiter	None (Anonymous)	Public Portfolio, Project Links, Resume Downloads, Arcade Games, Contact Form, Newsletter.	Cannot access <code>/admin/*</code> routes or mutate portfolio content.
Registered User / Player	JWT Bearer Token (User Scope)	Public Portfolio, Authenticated Game Leaderboard Submissions, Profile Preferences, Theme Customization.	Cannot access CMS editing controls or administrative user management.

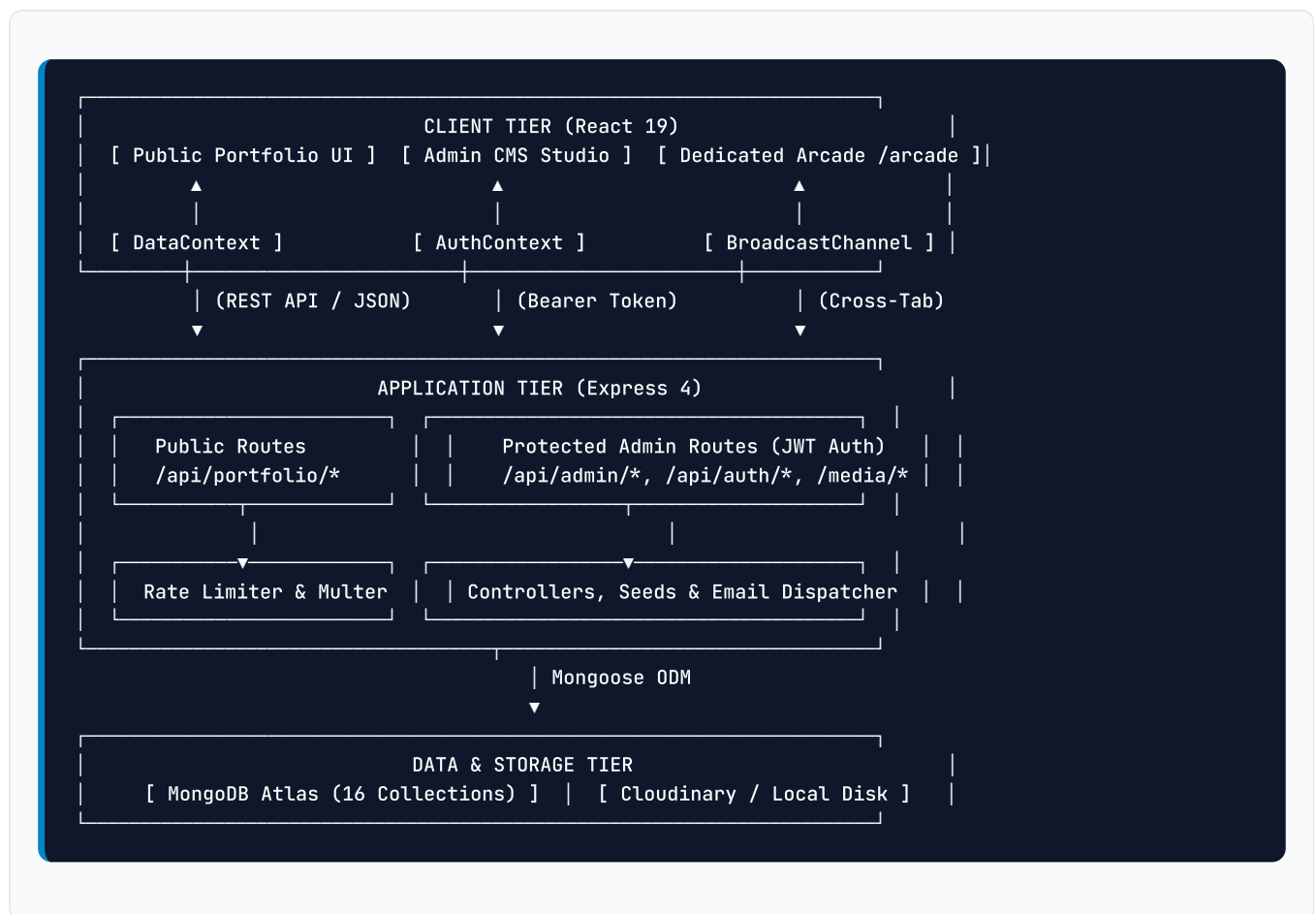
USER ROLE	AUTHENTICATION METHOD	ACCESSIBLE MODULES	SYSTEM RESTRICTIONS
Administrator (Admin)	JWT Bearer Token (Role = <code>admin</code>)	Full Admin Studio, CRUD over all 11 sections, User Management, Activity Logs, Documents Vault, Settings & Backups.	Full unconstrained privileges.

07 Technology Stack Specification

CATEGORY	TECHNOLOGY / PACKAGE	VERSION	PURPOSE IN PROJECT
Frontend Framework	React + React DOM	v19.2.4	Component-driven declarative UI rendering and virtual DOM state management.
Build Tool & Bundler	Vite	v8.0.4	Ultra-fast Hot Module Replacement (HMR) and optimized Rollup production bundling.
Client-side Routing	React Router DOM	v7.18.2	SPA routing, protected admin guards, dynamic URL parameter matching (<code>/arcade/:slug</code>).
Styling Architecture	Vanilla CSS Design Tokens	Modern CSS3	Custom CSS variables, glassmorphic <code>backdrop-filter</code> , Flexbox & CSS Grid.
Backend Runtime	Node.js	v18.0.0+	Asynchronous event-driven server runtime.
Web Framework	Express.js	v4.21.2	REST API endpoint architecture, middleware routing, and static file serving.
Database & ODM	MongoDB + Mongoose	v8.9.5	NoSQL document database with strict Schema typing, indexing, and pre-save hooks.
Authentication & Security	JSON Web Tokens (JWT) & BCrypt.js	JWT 9.0.2 / BCrypt 3.0.3	Stateless authentication tokens and salted password hashing (10 salt rounds).
Rate Limiting & Spam Protection	express-rate-limit	v7.5.0	IP-based window rate limiting on contact form submissions (5 requests per 10 minutes).
File Handling & Storage	Multer & Cloudinary SDK	Multer 1.4.5 / Cloudinary 2.5.1	Multipart form processing with dual storage: Memory upload / Local disk & Cloudinary.
Email Notification Service	Nodemailer	v6.10.0	SMTP client for automated email dispatch on contact form submissions.
PDF Generation Engine	Puppeteer Core + MS Edge	v25.7.0	Headless browser orchestration for publication-grade A4 documentation compilation.

08 System Architecture

The system employs a decoupled, three-tier client-server architecture. The frontend Single Page Application (SPA) communicates with the backend REST API service via asynchronous JSON HTTP requests over CORS:



09 Application Workflow

Visitor Interaction Flow

- Initial Hydration:** Client requests root route `/`. `DataContext` executes a single unified query to `GET /api/portfolio/public`.
- Dynamic Rendering:** React renders all sections using retrieved CMS document settings.
- Live Customization:** Visitor uses the draggable floating background widget to adjust visual themes locally in state.
- Resume Interaction:** Visitor downloads or previews resumes from the dynamic dropdown populated from the Resume Vault.
- Engagement & Gaming:** Playing arcade games triggers score recording via `POST /api/portfolio/games/:slug/score`.
- Contact Submission:** Sending a message passes through client validation and rate-limited dispatch.

Admin Studio Workflow

- Authentication:** Admin logs in via `/admin/login` against `POST /api/auth/login`.
- Token Verification:** Server issues an Access Token stored in `localStorage`.
- Live Editing:** Admin modifies content and saves via `PUT/POST` to `/api/admin/section/:type`.

4. **Instant Propagation:** Frontend broadcasts through `BroadcastChannel('portfolio_updates')`, updating public tabs instantly.

10 Project Directory Structure

```
Portfolio/
├─ public/
│   └─ docs/
├─ server/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── seeds/
│   ├── services/
│   └─ server.js
├─ src/
│   ├── admin/
│   │   ├── AdminLayout.jsx
│   │   ├── DashboardOverview.jsx
│   │   ├── NavbarCMS.jsx
│   │   ├── HeroCMS.jsx
│   │   ├── AboutCMS.jsx
│   │   ├── ProjectsCMS.jsx
│   │   ├── GamesCMS.jsx
│   │   ├── DocumentsCMS.jsx
│   │   ├── MessagesInbox.jsx
│   │   ├── UsersActivityCMS.jsx
│   │   └─ admin.css
│   ├── components/
│   │   ├── Navbar/
│   │   ├── Header/
│   │   ├── About/
│   │   ├── Projects/
│   │   ├── FunGame/
│   │   ├── Contact/
│   │   └─ Footer/
│   ├── context/
│   ├── config/
│   ├── App.jsx
│   ├── main.jsx
│   └─ index.css
├─ scripts/
│   └─ generate-pdf.js
├─ package.json
└─ vite.config.js

# Public static files & documents repository
# System documentation PDF repository
# Backend Express API & Database
# Database (db.js) & Cloudinary configurations
# Business logic controllers (admin, auth, portfolio, media)
# Security, JWT protection, and Multer file upload handlers
# 16 Mongoose Data Models (User, Hero, About, Project, Game, etc.)
# Express REST routes (adminRoutes, authRoutes, publicRoutes, mediaRoutes)
# Database auto-seeding scripts (seedData.js)
# Email notification service (emailService.js)
# Main Express application bootstrap entry point
# Frontend React 19 Application
# Protected Admin CMS Studio components
# Admin shell navigation, sidebar, and header
# Live telemetry, KPI counters, and activity stream
# Dedicated navigation styling & CTA customizer
# Hero layout switcher, typewriter roles, and badges
# Biography, stats, hobbies & Multi-Resume Vault
# Filterable project CRUD & order manager
# Arcade game settings, visibility & leaderboard manager
# Document vault & system report manager
# Contact message viewing & status management
# User directory, role assignment & audit logs
# Admin Studio design system & UI tokens
# Reusable Public UI Components
# Floating Dock, Cyber, Minimal, Full-Width navbar
# Multi-style Hero & Draggable Background Widget
# Bio, dynamic quick stats, hobbies & resume dropdowns
# Filterable projects grid & dynamic project cards
# Snake, 2048, Typing, Tic-Tac-Toe & Standalone Arcade
# Contact form, social triggers & rate limiter feedback
# Footer component, quick links & newsletter box
# React Context Hubs (AuthContext.jsx, DataContext.jsx)
# Frontend API resolution & media path utilities (api.js)
# Top-level route tree & protected route guards
# React 19 root DOM renderer
# Global theme tokens, typography & animations
# Automation & build scripts
# Puppeteer-based publication-grade PDF generator
# Project dependencies and npm scripts
# Vite configuration & proxy definitions
```

11 Frontend Architecture & Components

The frontend application is structured as a modular React 19 SPA powered by Vite. State is managed centrally through Context Providers without requiring bloated external state management libraries:

Core Context Providers

- `DataContext.jsx` : Single source of truth for public portfolio content with optimistic caching and `BroadcastChannel` synchronization.
- `AuthContext.jsx` : Manages authentication state, user profile preferences, token persistence in `localStorage` , and auth modal triggers.

Key Public UI Modules

COMPONENT	SOURCE DIRECTORY	KEY CAPABILITIES & INTERACTIVE FUNCTIONALITY
Navbar	<code>src/components/Navbar/</code>	Dynamic layout style switcher, animated logo pulse, live availability indicator pill, smooth section anchor scroll, and dark/light mode toggle.
Header (Hero)	<code>src/components/Header/</code>	Animated typewriter role text, dynamic tech badges, interactive Draggable Background Widget supporting 8 backdrop presets and live HSL color picking.
About & Vault	<code>src/components/About/</code>	Biography paragraphs, dynamic quick stats cards, interactive hobbies tags, and the Multi-Resume Vault with inline preview (↗) and direct download (↓).
Projects Grid	<code>src/components/Projects/</code>	Real-time category filtering (Full Stack, Frontend, Python, UI/UX, Open Source), live preview links, GitHub repository links, and star/fork metrics.
Arcade Lounge	<code>src/components/FunGame/</code>	Embedded mini-games (Snake, 2048, Typing Speed, Tic-Tac-Toe) with live WPM/Accuracy meters, Minimax AI engine, and global leaderboards.
Dedicated Arcade	<code>StandaloneArcadeWindow.jsx</code>	Dedicated full-screen distraction-free gaming environment at <code>/arcade</code> and <code>/arcade/:slug</code> .

12 Backend Architecture & Controllers

The backend is structured around Express REST controllers, Mongoose data models, and specialized security middleware:

- `portfolioController.js` : Handles public aggregated queries (`getPublicPortfolio`), contact message submission with Nodemailer dispatch, newsletter subscription, and game leaderboard submission/retrieval.
- `adminController.js` : Provides complete administrative CRUD operations across all portfolio sections, user directory management, status toggles, activity log inspection, and JSON backup/restore.
- `authController.js` : Implements secure user registration, BCrypt password validation, JWT access and refresh token generation, user preference updates, and avatar uploads.
- `mediaController.js` : Handles multipart asset uploads via Multer, routing to Cloudinary when configured or falling back to local disk storage.

13 API Specification

METHOD	ENDPOINT	AUTH	REQUEST PAYLOAD / PARAMS	SUCCESS RESPONSE	ERRORS
GET	/api/portfolio/public	Public	None	200: Aggregated JSON object with hero, about, navbar, projects, skills, games, etc.	500: Server Error
POST	/api/portfolio/contact	Public (Rate-limited)	{ name, email, subject, message }	201: { message: "Message sent successfully" }	400: Validation, 429: Too Many Requests
POST	/api/portfolio/subscribe	Public	{ email }	201: { message: "Subscribed successfully" }	400: Invalid Email, 409: Duplicate
POST	/api/portfolio/games/:slug/score	Public / Auth	{ playerName, score, metrics }	201: { message: "Score saved", scoreDoc }	400: Missing Fields, 500: Error
GET	/api/portfolio/games/:slug/leaderboard	Public	Query: ? limit=10	200: Array of top high score objects	500: Server Error
POST	/api/auth/login	Public	{ email, password }	200: { user, accessToken, refreshToken }	400: Missing fields, 401: Invalid Auth
GET	/api/admin/overview	Admin Bearer	None	200: KPI metrics, unread counts, recent activity logs	401: Unauthorized, 403: Forbidden
PUT	/api/admin/section/:type/:id	Admin Bearer	Updated Section Fields	200: Updated document object	401: Unauthorized, 404: Not Found
POST	/api/admin/upload/document	Admin Bearer	Multipart: file, title, category	201: Created Document Record	400: Invalid file, 401: Unauthorized

METHOD	ENDPOINT	AUTH	REQUEST PAYLOAD / PARAMS	SUCCESS RESPONSE	ERRORS
GET	/api/admin/backup/export	Admin Bearer	None	200: Complete JSON database snapshot	401: Unauthorized, 403: Forbidden

14 Database Architecture (MongoDB Schemas)

The database layer is implemented on MongoDB using Mongoose 8. The database utilizes 16 specialized schema collections with indexed fields for high-throughput queries:

User Schema (users)

- name (String, required)
- email (String, required, unique, indexed)
- password (String, hashed via bcrypt)
- role (Enum: 'admin' | 'editor' | 'user')
- preferences (darkMode, background, accentColor)
- lastLogin, loginCount, lastIp, device
- isSuspended (Boolean, default false)

GameScore Schema (gamescores)

- gameSlug (Enum: 'snake' | '2048' | 'typing' | 'tictactoe')
- playerName (String, default 'Player')
- score (Number, required, indexed)
- metrics (wpm, accuracy, highestTile, moves)
- userId (ObjectId, ref: 'User')
- Compound Index: { gameSlug: 1, score: -1 }

Hero Schema (heroes)

- layoutStyle (Enum: 'glassmorphism' | 'split' | 'editorial' | 'hologram')
- typewriterRoles (Array of Strings)
- techPills (Array of Strings)
- stats (Array of { value, label })
- socialLinks (Array of { label, href, icon })

About Schema & Vault (abouts)

- displayName, title, location, avatarUrl
- paragraphs (Array of Strings)
- quickStats, hobbies
- resumes (Array of { title, url, fileName, fileSize, isVisible, isDefault })

15 Authentication & Security Mechanisms

Security Architecture Summary

The system enforces a multi-layered security perimeter incorporating stateful cryptographic hashing, stateless JWT authorization, and defensive request throttling.

- Password Cryptography:** Passwords are never stored in plaintext. Mongoose pre-save hooks automatically hash passwords with **BCrypt using 10 salt rounds**.
- Dual-Token Authorization:** Upon successful login, the server issues a 7-day signed Access Token (`JWT_SECRET`) and a 30-day Refresh Token (`JWT_REFRESH_SECRET`).
- Role-Based Access Control (RBAC):** The `protectAdmin` middleware verifies token signature and confirms `user.role === 'admin'` before allowing access to administrative mutation routes.
- Spam Throttling:** The public contact form is protected by `express-rate-limit`, restricting clients to **5 message submissions per 10-minute window per IP**.
- File Upload Sanitization:** Multer enforces strict MIME-type allowlists (PDF, DOC, DOCX, JPEG, PNG, WEBP, SVG) and an upper file boundary of **10MB**.

16 Installation & Setup Guide

```
# 1. Clone the project repository
git clone https://github.com/Mahadebmaity/MCAWALLAH.git
cd Portfolio

# 2. Install frontend dependencies
npm install

# 3. Install backend dependencies
cd server
npm install
cd ..

# 4. Seed initial default database data
cd server
npm run seed
cd ..

# 5. Start development environment concurrently (Frontend + Backend)
npm run dev
```

17 Environment Variables Configuration

VARIABLE NAME	PURPOSE	EXAMPLE VALUE (PLACEHOLDER)
PORT	Backend HTTP server listening port	5000
NODE_ENV	Application runtime mode	development or production
MONGODB_URI	MongoDB connection string	mongodb://127.0.0.1:27017/portfolio_cms
JWT_SECRET	Secret key for signing access tokens	your_jwt_access_secret_key_here
JWT_EXPIRES_IN	Access token lifespan	7d
JWT_REFRESH_SECRET	Secret key for signing refresh tokens	your_jwt_refresh_secret_key_here
SMTP_HOST	SMTP server host for Nodemailer	smtp.gmail.com
SMTP_PORT	SMTP communication port	587
SMTP_USER	SMTP authenticated user account	your_email@gmail.com
SMTP_PASS	SMTP application password	your_smtp_app_password
CONTACT_RECEIVER_EMAIL	Destination inbox for contact messages	admin@portfolio.com

18 Testing Methodologies

- **Automated Test Suites (in `server/`):**
 - `test-auth.js` : Validates user registration, token generation, password verification, and RBAC rejection.
 - `test-api.js` : Tests public aggregated portfolio queries and error status code handling.
 - `test-games.js` : Verifies high score ingestion, leaderboard compound sorting, and metrics calculation.
 - `test-typewriter.js` : Validates role serialization and animated typewriter string arrays.
- **Manual End-to-End Testing:** Verified across multiple viewport widths (360px mobile to 4K displays), theme transitions, resume downloads, form error validations, and cross-tab BroadcastChannel state updates.

19 Error Handling Architecture

```
app.use((err, req, res, next) => {
  console.error('Server error:', err);
  res.status(err.status || 500).json({
    message: err.message || 'Internal Server Error',
    error: process.env.NODE_ENV === 'development' ? err.stack : undefined
  });
});
```

20 Screenshots & User Interface Documentation

1. Dynamic Floating Navigation Dock

Mac-style frosted glass capsule featuring active link detection, animated live availability badge, theme toggle, and customizable call-to-action button.

2. Multi-Style Hero Section

Showcases animated typewriter roles, quick stat pills, social triggers, and the draggable background customizer widget.

21 Performance Optimization

- **Single Aggregated Network Request:** All portfolio content is loaded in a single aggregated `GET /api/portfolio/public` query.
- **Compound Database Indexing:** Mongoose models use compound indices for sub-millisecond query responses.
- **Zero-Dependency Styling:** Clean Vanilla CSS design tokens without heavyweight styling framework overhead.

22 System Limitations

- **Single Primary Administrator Model:** Granular role-based permissions for editors are not yet fully differentiated.
- **Synchronous Nodemailer Dispatch:** Email notifications are processed synchronously.
- **Local Upload Fallback Dependency:** Relies on local disk storage when Cloudinary credentials are not configured.

23 Future Scope & Roadmap

Short-Term Enhancements

- OAuth2 Social Login integration (GitHub / Google SSO).
- Rich Text WYSIWYG Editor for blog posts and project case studies.
- Automated contact webhook notifications to Discord / Slack channels.

Long-Term Enhancements

- Interactive 3D WebGL / Three.js portfolio scene integration.
- AI-powered chatbot assistant for automated visitor inquiries and resume Q&A.
- Redis caching layer for ultra-high traffic scaling.

24 Deployment Guide

- **Frontend (Vercel / Netlify):** Configured via `vercel.json` for Single Page Application routing rewrites.
- **Backend (Render / Railway / Fly.io):** Deploy the `server/` folder as a Node.js web service.
- **Database (MongoDB Atlas):** Deploy an Atlas cluster and inject the connection string as `MONGODB_URI`.

25 Maintenance & Contribution Guide

1. **Fork & Branch:** Create a feature branch: `git checkout -b feature/amazing-feature`
2. **Code Standards:** Maintain ESLint standards (`npm run lint`).
3. **Testing:** Run backend test scripts before submitting PRs.

26 Version History & Changelog

VERSION	RELEASE DATE	KEY MILESTONES & CHANGES
v1.0.0	2026-08-17	Initial official production release with 11 CMS sections, Multi-Resume Vault, 4 Arcade Games, Live Telemetry, and Puppeteer PDF Generation Engine.

27 Conclusion

The **MCA WALLAH Portfolio & Dynamic CMS** delivers a complete, enterprise-grade solution that transforms personal portfolio management from a rigid, code-heavy chore into a dynamic, real-time administrative experience.

28 References & Documentation Links

- React 19 Documentation: <https://react.dev/>
- Vite Build Tool: <https://vitejs.dev/>
- Express.js Framework: <https://expressjs.com/>
- Mongoose ODM Documentation: <https://mongoosejs.com/>
- Puppeteer Documentation: <https://pptr.dev/>